

AI STUDENT SUPPORT ASSISTANT

Simple Agentic AI Mini Project

Prepared By	Abinaya A
Department	Computer Science Engineering
College	V.R.S. College of Engineering and Technology, Arasur
College Code	4227
Use Case	AI Student Support Assistant

1. Abstract

The **AI Student Support Assistant** is a simple web application designed to help college students get quick answers to common academic and campus-related questions. Instead of searching through many notices or documents, a student can type a question such as “What documents are needed for admission?” or “How do I contact the college office?”.

The project uses a small local knowledge base as its source of information. A basic retrieval step finds the most relevant information, a tool function can provide useful utilities such as office timings, and a small memory component remembers the student’s name during the current conversation. The design is intentionally simple so that a beginner can understand and explain every part.

This project demonstrates the basic idea of an agentic AI system without making the application unnecessarily advanced.

2. Introduction

Students frequently have questions about admission, syllabus, fees, departments, office timings, certificates, and campus facilities. The answers may already exist in notices, FAQs, or college information, but finding the correct information can take time.

This project provides one simple chat interface. The student enters a question, the application searches a small knowledge base, selects the closest answer, and displays it. If the question needs a simple utility, the application can call a tool. The application can also remember basic conversation information such as the student's name.

The project is not intended to replace college staff. It is a small demonstration project showing how retrieval, tools, and memory can work together in an AI assistant.

Example questions:

- “What documents are required for admission?”
- “When is the college office open?”
- “What is the CSE department about?”
- “My name is Ravi.” followed by “What is my name?”

3. Problem Statement

College students often need quick answers to repeated questions. Information may be spread across notices, websites, departments, and office instructions. A beginner-friendly assistant can reduce this search effort by keeping common information in one searchable knowledge base.

The problem addressed by this project is therefore: **How can a simple AI assistant answer common student questions from trusted college information while also using a small tool and short-term memory?**

The system should be easy to build, easy to test, and easy for a student team to explain during a project demonstration.

4. Objectives

- Create a simple web-based student support assistant.
- Store common college FAQs in a local knowledge base.
- Use a basic retrieval method to find relevant answers.
- Add one simple tool for college office timing / utility information.
- Add short-term conversation memory for the student's name.
- Provide a clean chat interface for students.
- Keep the project simple enough to complete in five days.
- Test the assistant with common and unknown questions.

5. Existing System

In a normal system, students may use different sources to find information.

Method	Typical issue
Ask college office	Depends on staff availability.
Search notices	Takes time to find the correct notice.
Search website	Information may be spread across pages.
Ask friends	Answer may be incomplete or outdated.

Drawback: The student must manually search multiple sources for repeated questions.

6. Proposed System

The proposed system provides a single chat screen. The user question goes to a simple controller. The controller checks whether the question matches a tool request. Otherwise it searches the local knowledge base and returns the most relevant answer. A small memory dictionary stores the student's name for the current session.

The project uses the three ideas shown in the training use case: **RAG + Tools + Memory**. Here, RAG is implemented in a very simple way using keyword/word-overlap retrieval instead of a complex vector database.

Simple flow:

Student → Chat UI → Agent Controller → Tool OR Knowledge Search → Answer → Student

7. Scope

Included in this version

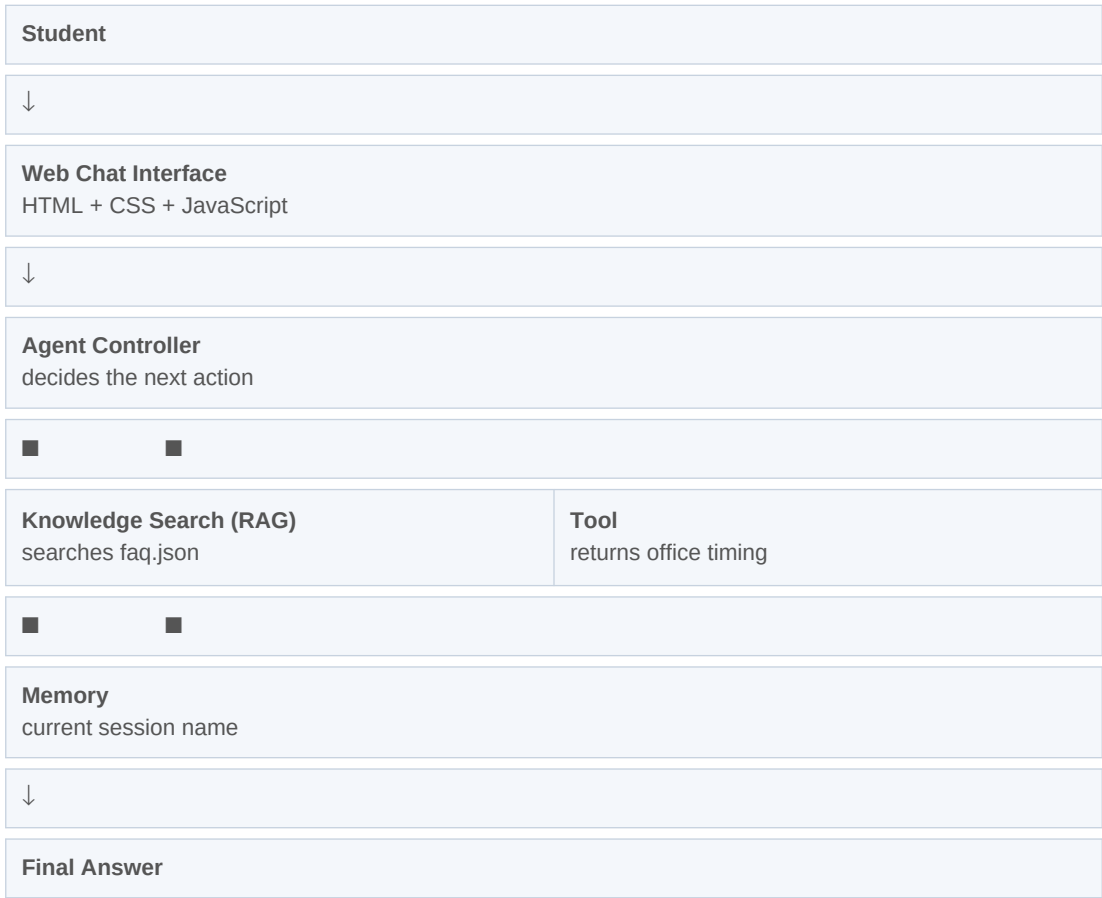
- Admission and general college FAQs.
- Department information.
- Basic office timing information.
- Simple student-name memory.
- Simple browser-based chat interface.
- Local data file, so internet is not required for the demo.

Not included in this version

- Real-time college database connection.
- Automatic fee payment or admission actions.
- Complex large-language-model training.
- Private student records.
- Production-level authentication and security.

8. Architecture

The architecture is intentionally small and easy to understand.



9. Detailed Procedure

Step 1 — Start the application

Run the Flask server and open the project in a browser.

Step 2 — Enter a question

The student types a normal college-related question.

Step 3 — Agent checks the request

The controller checks memory and simple tool conditions first.

Step 4 — Tool or retrieval

For office timing, the tool is called. For normal FAQs, the local knowledge base is searched.

Step 5 — Select answer

The FAQ with the highest word-overlap score is selected.

Step 6 — Memory

If the student says “My name is ...”, the name is stored in the current session.

Step 7 — Display

The answer is returned to the browser and shown in the chat.

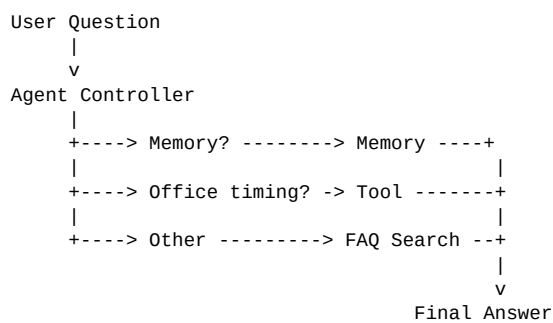
10. Module Explanation

Module	Purpose
1. Chat UI	Takes the question and displays the answer.
2. Agent Controller	Decides whether to use memory, a tool, or retrieval.
3. Knowledge Base	Stores common student FAQs in JSON.
4. Retrieval	Finds the closest FAQ using simple word overlap.
5. Tool	Returns basic office timing information.
6. Memory	Stores the student's name during the current session.

11. Algorithm / Workflow

- Receive the user question.
- Convert the question into lowercase words.
- Check whether the user is providing a name.
- If the user asks for their name, read it from memory.
- If the question asks for office timing, call the tool.
- Otherwise compare the question with the FAQ knowledge base.
- Select the highest scoring FAQ.
- If no useful match exists, show a fallback message.
- Return the answer to the browser.

Workflow



12. Technology Stack

Technology	Use
Python	Main programming language
Flask	Small web server and API
HTML	Chat page structure
CSS	Simple interface styling
JavaScript	Sends questions to Flask
JSON	Local FAQ knowledge base
Browser	Runs the user interface

This simple demo does not require a paid AI API. It can be demonstrated locally.

13. Folder Structure

```
student-support-assistant/  
■  
■■■ app.py  
■■■ faq.json  
■■■ requirements.txt  
■  
■■■ templates/  
■   ■■■ index.html  
■  
■■■ static/  
    ■■■ style.css  
    ■■■ script.js
```

- app.py — server and agent logic.
- faq.json — FAQ knowledge base.
- requirements.txt — dependency list.
- templates/index.html — chat page.
- static/style.css — page styling.
- static/script.js — browser-side chat logic.

14. Installation Procedure

Requirement: Python 3.10+ and a web browser.

1. Create the project folder.
2. Open Terminal / Command Prompt in the folder.
3. Create a virtual environment:

```
python -m venv venv
```

4. Activate it.

```
Windows:  
venv\Scripts\activate
```

```
Linux / macOS:  
source venv/bin/activate
```

5. Install Flask:

```
pip install -r requirements.txt
```

6. Start the project:

```
python app.py
```

7. Open:

```
http://127.0.0.1:5000
```

15. Sample Inputs / Outputs

Sample Input	Expected Output
What documents are needed for admission?	The assistant lists basic admission documents.
When is the college office open?	Office timing: 9:00 AM to 5:00 PM, Monday to Friday.
What is CSE?	CSE means Computer Science and Engineering.
My name is Arun.	Nice to meet you, Arun! I will remember your name during this session.
What is my name?	Your name is Arun.
Tell me about the moon.	Sorry, I do not have that information in my college knowledge base.

16. Test Cases

TC	Test	Expected	Result
TC01	Open home page	Chat page loads	Pass
TC02	Ask known FAQ	Relevant answer shown	Pass
TC03	Ask office timing	Tool answer shown	Pass
TC04	Save name	Name stored	Pass
TC05	Ask stored name	Correct name returned	Pass
TC06	Unknown question	Fallback shown	Pass
TC07	Empty input	Blank request rejected	Pass

17. 5-Day Development Procedure

Day	Work completed
Day 1	Select use case, define problem/objectives and collect FAQs.
Day 2	Create Flask project, JSON knowledge base and retrieval logic.
Day 3	Add agent controller, tool and simple memory.
Day 4	Build chat UI and connect it to the Flask API.
Day 5	Test, fix errors, prepare demo, report and presentation.

18. Advantages

- Simple and beginner-friendly.
- One place for common student questions.
- Fast response for known FAQs.
- Demonstrates RAG, tool calling and memory.
- Low-cost and suitable for a local demo.
- Easy to extend by adding more FAQs.
- Easy to explain during viva.

19. Limitations

- Knowledge base is small and manually maintained.
- Simple word-overlap retrieval is limited.
- Unknown questions cannot be answered reliably.
- Memory is only for the current session.
- No real college database or authentication is connected.
- Stored answers must be kept updated.

20. Future Enhancements

- Use embeddings and a vector database for better semantic RAG.
- Connect a modern LLM for natural conversation.
- Connect the official college website and notices.
- Add secure student login.
- Add Tamil + English support.
- Allow college PDF/notice uploads.
- Add voice input and output.
- Add an admin page for updating FAQs.

21. Conclusion

The AI Student Support Assistant is a small and complete demonstration of an agentic AI use case. It combines a knowledge base, simple retrieval, a tool and short-term memory in one web application.

The project is intentionally normal and beginner-friendly rather than highly advanced. It can be completed in five days and explained easily during a project review or viva.

Because the design is modular, the simple retrieval method can later be replaced by a stronger RAG system and the tool can later connect to real college data.

22. Full Source Code Appendix

Create the files using the folder structure shown in Section 13.

A. app.py

```
from flask import Flask, render_template, request, jsonify, session
import json
import re

app = Flask(__name__)
app.secret_key = "simple-demo-secret"

with open("faq.json", "r", encoding="utf-8") as f:
    FAQs = json.load(f)

def words(text):
    return set(re.findall(r"[a-zA-Z0-9]+", text.lower()))

def retrieve_answer(question):
    q_words = words(question)
    best_score = 0
    best_answer = None

    for item in FAQs:
        item_words = words(item["question"] + " " + item["answer"])
        score = len(q_words & item_words)
        if score > best_score:
            best_score = score
            best_answer = item["answer"]

    return best_answer if best_score > 0 else None

def office_tool():
    return "College office timing: 9:00 AM to 5:00 PM, Monday to Friday."

def agent(question):
    q = question.strip()
    low = q.lower()

    match = re.search(r"my name is ([a-zA-Z ]+)", q, re.I)
    if match:
        name = match.group(1).strip().title()
        session["student_name"] = name
        return f"Nice to meet you, {name}! I will remember your name during this session."

    if "what is my name" in low or "do you know my name" in low:
        name = session.get("student_name")
        return f"Your name is {name}." if name else "You have not told me your name yet."

    if "office" in low and ("time" in low or "timing" in low or "open" in low):
        return office_tool()

    answer = retrieve_answer(q)
    if answer:
        return answer

    return ("Sorry, I do not have that information in my college "
           "knowledge base. Please check with the college office.")

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
    question = data.get("message", "")

    if not question.strip():
        return jsonify({"answer": "Please type a question."})

    return jsonify({"answer": agent(question)})

if __name__ == "__main__":
    app.run(debug=True)
```

B. faq.json

```
[
  {
    "question": "What documents are needed for admission?",
    "answer": "Basic admission documents may include ID proof, 10th and 12th certificates, transfer certificate, community c
  },
  {
    "question": "What is CSE?",
    "answer": "CSE means Computer Science and Engineering. It covers programming, software development, databases, computer
  },
  {
    "question": "What is the college office?",
    "answer": "The college office handles common administrative work such as certificates, student records, applications and
  },
  {
    "question": "What are the library timings?",
    "answer": "For this demo, the library is assumed to be open from 9:00 AM to 5:00 PM on working days. Replace this with y
  },
  {
    "question": "How can I contact the college?",
    "answer": "Students should use the official college phone number, email address or office counter listed by the college.
  },
  {
    "question": "What is a syllabus?",
    "answer": "A syllabus is the list of subjects and topics that students study in a course or semester."
  },
  {
    "question": "What is attendance?",
    "answer": "Attendance is the record of the classes a student attends. Students should follow the attendance rules of the
  },
  {
    "question": "What is a department?",
    "answer": "A department is an academic unit of a college that manages a particular course or subject area, such as CSE,
  }
]
```

C. requirements.txt

```
Flask==3.1.2
```

D. templates/index.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>AI Student Support Assistant</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <div class="container">
    <h1>AI Student Support Assistant</h1>
    <p class="sub">Ask a college-related question</p>

    <div id="chat"></div>

    <div class="input-row">
      <input id="message" placeholder="Type your question..." />
      <button onclick="sendMessage()">Send</button>
    </div>
  </div>

  <script src="/static/script.js"></script>
</body>
</html>
```


E. static/style.css

```
body {
  font-family: Arial, sans-serif;
  background: #f2f5f8;
  margin: 0;
}

.container {
  width: 90%;
  max-width: 700px;
  margin: 40px auto;
  background: white;
  padding: 25px;
  border-radius: 12px;
  box-shadow: 0 2px 10px #cccccc;
}

h1 { margin-bottom: 5px; }
.sub { color: #666; }

#chat {
  min-height: 350px;
  border: 1px solid #ddd;
  padding: 15px;
  margin: 20px 0;
  border-radius: 8px;
  background: #fafafa;
}

.msg {
  padding: 10px;
  margin: 8px 0;
  border-radius: 8px;
}

.user {
  background: #e5f0ff;
  text-align: right;
}

.bot { background: #eeeeee; }

.input-row {
  display: flex;
  gap: 8px;
}

input {
  flex: 1;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 6px;
}

button {
  padding: 12px 18px;
  border: 0;
  border-radius: 6px;
  cursor: pointer;
}
```

F. static/script.js

```
async function sendMessage() {
  const input = document.getElementById("message");
  const text = input.value.trim();

  if (!text) return;

  addMessage(text, "user");
  input.value = "";

  const response = await fetch("/chat", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({message: text})
  });

  const data = await response.json();
  addMessage(data.answer, "bot");
}

function addMessage(text, type) {
  const chat = document.getElementById("chat");
  const div = document.createElement("div");

  div.className = "msg " + type;
  div.textContent = text;

  chat.appendChild(div);
  chat.scrollTop = chat.scrollHeight;
}

document.getElementById("message").addEventListener("keydown", function(e) {
  if (e.key === "Enter") sendMessage();
});
```

23. Source Code Concept Mapping

RAG / Retrieval: *retrieve_answer()* searches the local FAQ data and selects the closest stored answer.

Tool Calling: *office_tool()* is selected when the student asks about office timing.

Memory: Flask session storage keeps the student's name during the current session.

Agent: *agent()* decides whether to use memory, the tool, or FAQ retrieval.

End of Project Report